

Hands on 1

Create a Spring Web Project using Maven

Follow steps below to create a project:

1. Go to <https://start.spring.io/>
2. Change Group as “com.cognizant”
3. Change Artifact Id as “spring-learn”
4. Select Spring Boot DevTools and Spring Web
5. Create and download the project as zip
6. Extract the zip in root folder to Eclipse Workspace
7. Build the project using ‘mvn clean package -
Dhttp.proxyHost=proxy.cognizant.com -Dhttp.proxyPort=6050 -
Dhttps.proxyHost=proxy.cognizant.com -Dhttps.proxyPort=6050 -
Dhttp.proxyUser=123456’ command in command line
8. Import the project in Eclipse "File > Import > Maven > Existing Maven Projects > Click Browse and select extracted folder > Finish"
9. Include logs to verify if main() method of SpringLearnApplication.
10. Run the SpringLearnApplication class.

SME to walk through the following aspects related to the project created:

1. src/main/java - Folder with application code
2. src/main/resources - Folder for application configuration
3. src/test/java - Folder with code for testing the application
4. SpringLearnApplication.java - Walkthrough the main() method.
5. Purpose of @SpringBootApplication annotation
6. pom.xml
 1. Walkthrough all the configuration defined in XML file
 2. Open 'Dependency Hierarchy' and show the dependency tree.

The screenshot shows the Spring Initializr web application. On the left, there is a sidebar with a hamburger menu icon and a refresh icon. The main content area is divided into several sections:

- Project:** Radio buttons for `Gradle - Groovy`, `Gradle - Kotlin`, and `Maven` (selected).
- Language:** Radio buttons for `Java` (selected), `Kotlin`, and `Groovy`.
- Spring Boot:** Radio buttons for `4.1.1 (SNAPSHOT)`, `4.1.0` (selected), `4.0.8 (SNAPSHOT)`, and `4.0.7`. Below it is a radio button for `3.5.16`.
- Project Metadata:** Text input fields for `Group` (containing `com.cognizant`), `Artifact` (containing `spring-learn`), and `Package name` (containing `spring-learn`).
- Packaging:** Radio buttons for `Jar` (selected) and `War`.
- Configuration:** Radio buttons for `Properties` (selected) and `YAML`.
- Java:** Radio buttons for `26`, `25`, `21` (selected), and `17`.
- Dependencies:** A button labeled `ADD DEPENDENCIES... CTRL + B`. Below it, two sections are visible:
 - Spring Web:** Labeled `WEB`. Description: "Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container."
 - Spring Boot DevTools:** Labeled `DEVELOPER TOOLS`. Description: "Provides fast application restarts, LiveReload, and configurations for enhanced development experience."

Project Structure

src/main/java

Contains the Java source code of the application.

src/main/resources

Contains configuration files such as `application.properties`.

src/test/java

Contains test classes used for unit testing.

SpringLearnApplication.java

The main class of the Spring Boot application. It contains the `main()` method, which starts the application using `SpringApplication.run()`.

pom.xml

The Maven configuration file that contains project information, dependencies, plugins, and build configuration.

Purpose of @SpringBootApplication

`@SpringBootApplication` is the main annotation used in Spring Boot. It combines:

- `@Configuration`
- `@EnableAutoConfiguration`
- `@ComponentScan`

It enables auto-configuration, component scanning, and Java-based configuration for the application.

Logging Code

```
package spring_learn;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SpringLearnApplication {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(SpringLearnApplication.class);

    public static void main(String[] args) {

        LOGGER.info("START");

        SpringApplication.run(SpringLearnApplication.class, args);

        LOGGER.info("END");
    }
}
```

Output

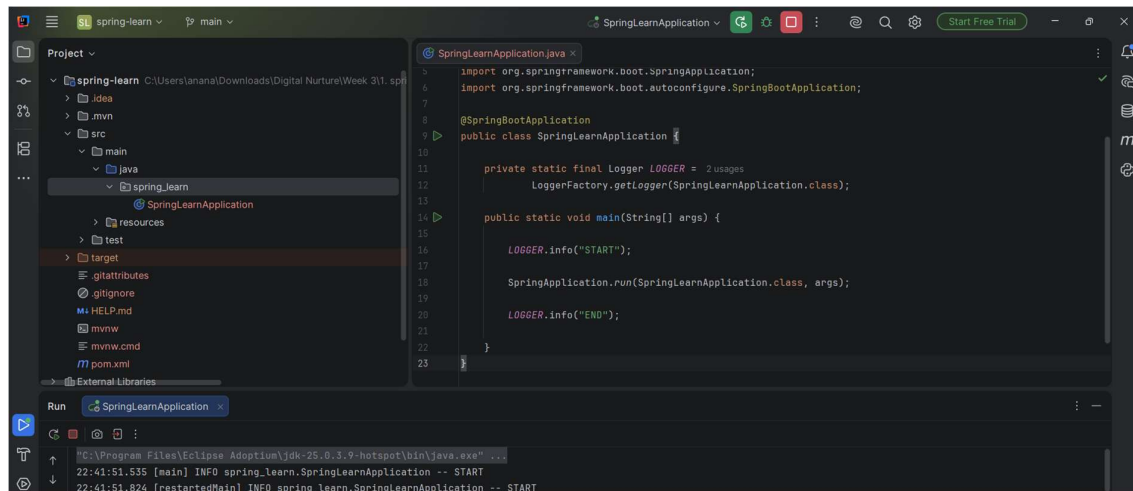
INFO SpringLearnApplication -- START

:: Spring Boot ::

Tomcat started on port(s): 8080

Started SpringLearnApplication

INFO SpringLearnApplication -- END



Result

A Spring Boot Web project was successfully created using Maven. The application was executed successfully, and the logging statements verified the execution of the main() method.

Hands on 4

Spring Core – Load Country from Spring Configuration XML

An airlines website is going to support booking on four countries. There will be a drop down on the home page of this website to select the respective country. It is also important to store the two-character ISO code of each country.

Code	Name
US	United States
DE	Germany
IN	India
JP	Japan

Above data has to be stored in spring configuration file. Write a program to read this configuration file and display the details.

Steps to implement

- Pick any one of your choice country to configure in Spring XML configuration named country.xml.
- Create a bean tag in spring configuration for country and set the property and values

```
<bean id="country" class="com.cognizant.springlearn.Country">
```

```
  <property name="code" value="IN" />
```

```
  <property name="name" value="India" />
```

```
</bean>
```

- Create Country class with following aspects:
 - Instance variables for code and name
 - Implement empty parameter constructor with inclusion of debug log within the constructor with log message as “Inside Country Constructor.”

- **Generate getters and setters with inclusion of debug with relevant message within each setter and getter method.**
- **Generate toString() method**
- **Create a method displayCountry() in SpringLearnApplication.java, which will read the country bean from spring configuration file and display the country details. ClassPathXmlApplicationContext, ApplicationContext and context.getBean("beanId", Country.class). Refer sample code for displayCountry() method below.**

ApplicationContext context = new

ClassPathXmlApplicationContext("country.xml");

Country country = (Country) context.getBean("country", Country.class);

LOGGER.debug("Country : {}", country.toString());

- **Invoke displayCountry() method in main() method of SpringLearnApplication.java.**
- **Execute main() method and check the logs to find out which constructors and methods were invoked.**

SME to provide more detailing about the following aspects:

- **bean tag, id attribute, class attribute, property tag, name attribute, value attribute**
- **ApplicationContext, ClassPathXmlApplicationContext**
- **What exactly happens when context.getBean() is invoked**

Aim

To load a Country bean from a Spring XML configuration file using the Spring IoC container and display the country details.

Procedure

1. Create a Country class with code and name properties.
 2. Add an empty constructor with a debug log.
 3. Generate getters and setters with debug logs.
 4. Override the toString() method.
 5. Create country.xml in the resources folder.
 6. Configure the country bean with code and name properties.
 7. Read the bean using ClassPathXmlApplicationContext.
 8. Display the country details using LOGGER.debug().
 9. Run the application and verify the logs.
-

Files Created

Country.java

- Contains instance variables:
 - code
 - name
 - Includes:
 - Default constructor
 - Getters
 - Setters
 - toString()
-

country.xml

```
<bean id="country" class="spring_learn.Country">  
  <property name="code" value="IN"/>
```

```
<property name="name" value="India"/>
</bean>
```

SpringLearnApplication.java

Uses:

```
ApplicationContext context =
    new ClassPathXmlApplicationContext("country.xml");
```

```
Country country =
    context.getBean("country", Country.class);
```

```
LOGGER.debug("Country : {}", country.toString());
```

Output

Inside Country Constructor.

Inside setCode()

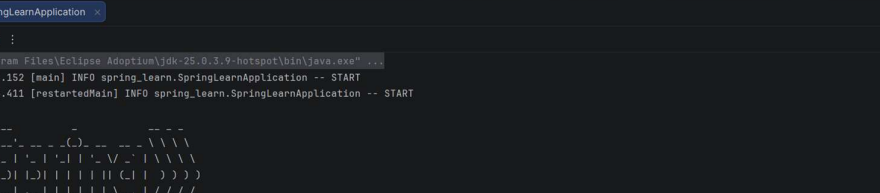
Inside setName()

Country : Country [code=IN, name=India]

(Attach your console screenshot if required.)

Result

The Country bean was successfully loaded from the Spring XML configuration file using ClassPathXmlApplicationContext, and the country details were displayed successfully.



```
Project > nom.xml (spring-learn) > Country.java > country.xml > SpringLearnApplication.java > application.properties
Run > SpringLearnApplication
C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe ...
22:49:56.152 [main] INFO spring_learn.SpringLearnApplication -- START
22:49:56.411 [restartedMain] INFO spring_learn.SpringLearnApplication -- START

  \ \ / _ _ _ _ _ _ _ _ _ _ \ \ / _ _ _ _ _
 ( ( ) ( ) ( ) ( ) ( ) ( ) ( ) ( ) ( ) ( )
  W _ _ _ _ _ _ _ _ _ _ W _ _ _ _ _
  ' _ _ _ _ _ _ _ _ _ _ ' _ _ _ _ _
  =====|=====|=====|=====|=====

:: Spring Boot ::                (v4.1.0)

2026-07-02T22:49:56.852+05:30 INFO 18496 --- [spring-learn] [ restartedMain] spring_learn.SpringLearnApplication : Starting SpringLearnApplication using Java 25.0.3 with
2026-07-02T22:49:56.854+05:30 DEBUG 18496 --- [spring-learn] [ restartedMain] spring_learn.SpringLearnApplication : Running with Spring Boot v4.1.0, Spring v7.0.8
2026-07-02T22:49:56.855+05:30 INFO 18496 --- [spring-learn] [ restartedMain] spring_learn.SpringLearnApplication : No active profile set, falling back to 1 default profile: 'default'
2026-07-02T22:49:56.914+05:30 INFO 18496 --- [spring-learn] [ restartedMain] o.s.DevTools.PropertyDefaultsPostProcessor : DevTools property defaults active! Set 'spring.devtools
2026-07-02T22:49:56.914+05:30 INFO 18496 --- [spring-learn] [ restartedMain] o.s.DevTools.PropertyDefaultsPostProcessor : For additional web related logging consider setting '
2026-07-02T22:49:57.636+05:30 INFO 18496 --- [spring-learn] [ restartedMain] o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-07-02T22:49:57.651+05:30 INFO 18496 --- [spring-learn] [ restartedMain] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-07-02T22:49:57.651+05:30 INFO 18496 --- [spring-learn] [ restartedMain] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/11.0.22]
2026-07-02T22:49:57.681+05:30 INFO 18496 --- [spring-learn] [ restartedMain] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed
2026-07-02T22:49:58.021+05:30 INFO 18496 --- [spring-learn] [ restartedMain] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path
2026-07-02T22:49:58.028+05:30 INFO 18496 --- [spring-learn] [ restartedMain] spring_learn.SpringLearnApplication : Started SpringLearnApplication in 1.61 seconds (process
2026-07-02T22:49:58.233+05:30 DEBUG 18496 --- [spring-learn] [ restartedMain] spring_learn.Country : Inside Country Constructor.
2026-07-02T22:49:58.234+05:30 DEBUG 18496 --- [spring-learn] [ restartedMain] spring_learn.Country : Inside setName()
2026-07-02T22:49:58.235+05:30 DEBUG 18496 --- [spring-learn] [ restartedMain] spring_learn.Country : Country : Country [code=IN, name=India]
2026-07-02T22:49:58.236+05:30 INFO 18496 --- [spring-learn] [ restartedMain] spring_learn.SpringLearnApplication : END
```